

PLATAFORMA ZIPLOC SAS
Sistema de Billeteras Digitales y Analítica de Transacciones
Informe Técnico Final

Presentado por:

Juan Esteban Galeano Bolaños

CC: 1005087822

Juan Steban Acosta Ramirez

CC: 1090273599

Asignatura: Estructuras de Datos

Fecha: 24/05/2026

01. Introducción

La plataforma ZIPLOC SAS corresponde a un sistema fintech académico desarrollado mediante Spring Boot, cuyo propósito principal es simular el funcionamiento de una aplicación de billeteras digitales modernas. El proyecto integra funcionalidades relacionadas con la administración de usuarios, gestión de billeteras electrónicas, procesamiento de transacciones financieras, operaciones programadas, generación de recompensas, notificaciones y servicios de analítica orientados a la detección de comportamientos financieros inusuales.

El desarrollo de este sistema permite demostrar la aplicación práctica de estructuras de datos en un entorno cercano a la realidad empresarial. En la actualidad, las plataformas financieras requieren rapidez en el procesamiento de información, seguridad en las operaciones y capacidad analítica para interpretar grandes volúmenes de datos. Por ello, la correcta selección de estructuras computacionales se convierte en un elemento fundamental para garantizar eficiencia, escalabilidad y confiabilidad dentro del sistema.

Módulos principales

- api.controller: endpoints REST y manejo uniforme de errores.
- api.dto: objetos de entrada/salida de la API.
- application.service: reglas de negocio y coordinación de estructuras.
- domain.model: entidades de dominio en memoria.
- domain.enums: estados, tipos y niveles usados por la plataforma.
- domain.structures: implementaciones propias de listas, pila, colas, árbol, tablas hash y grafo.

Endpoints base.

El contexto de la API es /api.

- /api/usuarios: registrar, listar, modificar, eliminar y consultar ranking por puntos.
- /api/billeteras: crear, listar, actualizar, eliminar y consultar saldo.
- /api/transacciones: recargar, retirar, transferir, consultar historial y revertir.
- /api/programadas: programar y procesar operaciones automáticas.

- /api/recompensas: consultar puntos, nivel y canjear beneficios.
- /api/notificaciones: listar, despachar y marcar alertas.
- /api/analitica: reportes, top de uso, auditoria, grafo, frecuencia y rendimiento.

Ejecutar

```
$env:JAVA_HOME='C:\Users\Luich\.jdk\openjdk-26.0.1'
```

```
$env:Path="$env:JAVA_HOME\bin;$env:Path"
```

```
cmd /c mvnw.cmd spring-boot:run
```

Probar

```
$env:JAVA_HOME='C:\Users\Luich\.jdk\openjdk-26.0.1'
```

```
$env:Path="$env:JAVA_HOME\bin;$env:Path"
```

```
cmd /c mvnw.cmd test
```

02. Contexto del Problema

Las plataformas fintech actuales deben enfrentar múltiples desafíos asociados con la administración masiva de datos financieros y la necesidad de mantener operaciones rápidas y seguras. Los sistemas de billeteras digitales requieren almacenar información de usuarios, movimientos monetarios, historiales de transacciones y patrones de comportamiento financiero en tiempo real.

Además, existe una creciente necesidad de implementar mecanismos de detección de fraude y monitoreo inteligente que permitan identificar operaciones sospechosas, transacciones inusuales o actividades fuera de los parámetros normales. Bajo este contexto, ZIPLOC SAS surge como una solución académica que modela el comportamiento de una plataforma financiera moderna utilizando estructuras de datos eficientes y arquitectura por capas.

03. Objetivos

Objetivo

General

Desarrollar una plataforma fintech eficiente mediante la implementación de estructuras de datos y una arquitectura modular orientada a la administración de billeteras digitales y análisis financiero.

Objetivos

Específicos

- Implementar estructuras de datos como listas, pilas, colas, árboles, tablas hash y grafos.
- Diseñar un modelo de dominio CORE organizado y escalable.
- Aplicar arquitectura por capas utilizando Spring Boot.
- Integrar servicios de analítica y detección de fraude.
- Justificar técnicamente el uso de cada estructura de datos.
- Simular procesos financieros reales relacionados con transacciones digitales.

04. Arquitectura del Sistema

El sistema implementa una arquitectura por capas que permite separar responsabilidades y garantizar modularidad en el desarrollo. La capa Controller se encarga de gestionar los endpoints REST y procesar las solicitudes realizadas por los clientes. La capa Service contiene la lógica de negocio y las validaciones necesarias para el funcionamiento del sistema.

Por otro lado, la clase PlataformaContext centraliza las estructuras de datos utilizadas dentro del sistema, incluyendo tablas hash para usuarios y billeteras, gestores de transacciones, colas de operaciones programadas, árboles de fidelización y grafos de transferencias.

El flujo principal del sistema sigue la secuencia Cliente → Controller → Service → Estructuras → Respuesta, permitiendo mantener un diseño organizado y fácil de mantener. Esta arquitectura mejora la coherencia interna del sistema y facilita la escalabilidad futura.

La aplicación usa una arquitectura por capas sencilla para mantener separadas las responsabilidades del proyecto académico.

classDiagram

```
class Controller {
```

```
+recibe HTTP
+retorna ApiResponse
}
class Service {
    +valida reglas
    +orquestra estructuras
}
class PlataformaContext {
    +TablaUsuarios
    +TablaBilleteras
    +GestorTransacciones
    +ColaProgramadas
    +GrafoTransferencias
    +ArbolFidelizacion
}
class DomainModel {
    +Usuario
    +Billetera
    +Transaccion
    +OperacionProgramada
    +Alerta
    +Beneficio
    +EventoAuditoria
}
```

Controller --> Service

Service --> PlataformaContext

Service --> DomainModel

PlataformaContext --> DomainModel

FLUJO DE TRANSFERENCIA

1. TransaccionController recibe una solicitud en /api/transacciones/transferir.
2. TransaccionService valida billeteras, saldo y estado.
3. BilleteraService descuenta saldo de origen y suma saldo al destino.
4. GestorTransacciones registra el historial y la pila de reversion.
5. GrafoTransferencias registra la relacion entre usuarios.
6. SistemaRecompensas calcula puntos y actualiza nivel del usuario.
7. TransaccionService evalua riesgo y registra auditoria si aplica.
8. NotificacionService conserva alertas recientes por usuario.

DECISIONES

- La persistencia es en memoria porque el objetivo del proyecto es demostrar estructuras de datos, no una base de datos.
- Los servicios son @Service de Spring para conservar una API REST limpia.
- PlataformaContext centraliza las estructuras compartidas y evita duplicar estado entre servicios.
- Las validaciones de negocio lanzan excepciones controladas por ApiExceptionHandler.

05. Dominio CORE

El dominio CORE representa la estructura central del sistema ZIPLOC SAS y modela el funcionamiento principal de la plataforma financiera basada en billeteras digitales. Este dominio define las entidades fundamentales, sus relaciones y las reglas principales que permiten administrar usuarios, billeteras, transacciones y operaciones financieras dentro del sistema.

La entidad principal corresponde al Usuario, quien representa a cada cliente registrado dentro de la plataforma. Cada usuario almacena información básica como nombre, correo electrónico, teléfono, fecha de registro y nivel de clasificación. Adicionalmente, el sistema mantiene un control de puntos acumulados que posteriormente son utilizados por el módulo de recompensas y fidelización. El usuario constituye el núcleo de interacción dentro de la plataforma, ya que todas las operaciones financieras y procesos analíticos se relacionan directamente con su actividad.

Cada usuario puede poseer una o varias Billeteras digitales asociadas a diferentes objetivos financieros. Estas billeteras permiten organizar fondos según categorías como ahorro, gastos, compras o cuentas corrientes. Cada billetera administra información relacionada con saldo disponible, estado de activación y tipo de cuenta, permitiendo realizar operaciones financieras básicas como recargas, retiros y consultas de saldo.

La relación entre Usuario y Billetera establece una estructura flexible y escalable, ya que un mismo usuario puede administrar múltiples fuentes de dinero dentro de la plataforma. Esta organización facilita la segmentación de gastos y mejora el control financiero por parte del cliente.

Por otra parte, las Transacciones representan cada movimiento financiero realizado dentro del sistema. Estas operaciones almacenan información relacionada con fecha, monto, tipo de transacción, billeteras involucradas, estado de ejecución y nivel de riesgo asociado. Las transacciones permiten mantener trazabilidad completa sobre todas las operaciones financieras realizadas en la plataforma, garantizando control, auditoría y consistencia en los movimientos monetarios.

El sistema clasifica las transacciones mediante diferentes estados como pendiente, completada, revertida o rechazada. Esto permite administrar correctamente el ciclo de vida de cada operación y controlar posibles errores, cancelaciones o validaciones de seguridad. Asimismo, cada transacción puede asociarse a un nivel de riesgo determinado, permitiendo que los servicios de analítica y monitoreo identifiquen comportamientos sospechosos o actividades fuera de lo habitual.

El dominio también incorpora la entidad OperacionProgramada, diseñada para automatizar procesos financieros recurrentes o futuros. Este componente permite agendar transferencias, pagos o movimientos automáticos con fechas específicas y niveles de prioridad determinados. Gracias a ello, el sistema puede simular funcionalidades comunes en plataformas bancarias modernas relacionadas con pagos automáticos y programación financiera.

Las operaciones programadas contienen información asociada a fecha de ejecución, billeteras de origen y destino, prioridad, valor de la operación y estado de ejecución. Además, incluyen métodos

que permiten ejecutar o cancelar automáticamente operaciones dependiendo de las condiciones definidas por el usuario.

Complementando el modelo principal, el sistema utiliza diferentes enumeraciones orientadas a garantizar organización y estandarización dentro de la lógica del negocio. Entre ellas se encuentran NivelUsuario, TipoBilletera, EstadoTransaccion y NivelRiesgo.

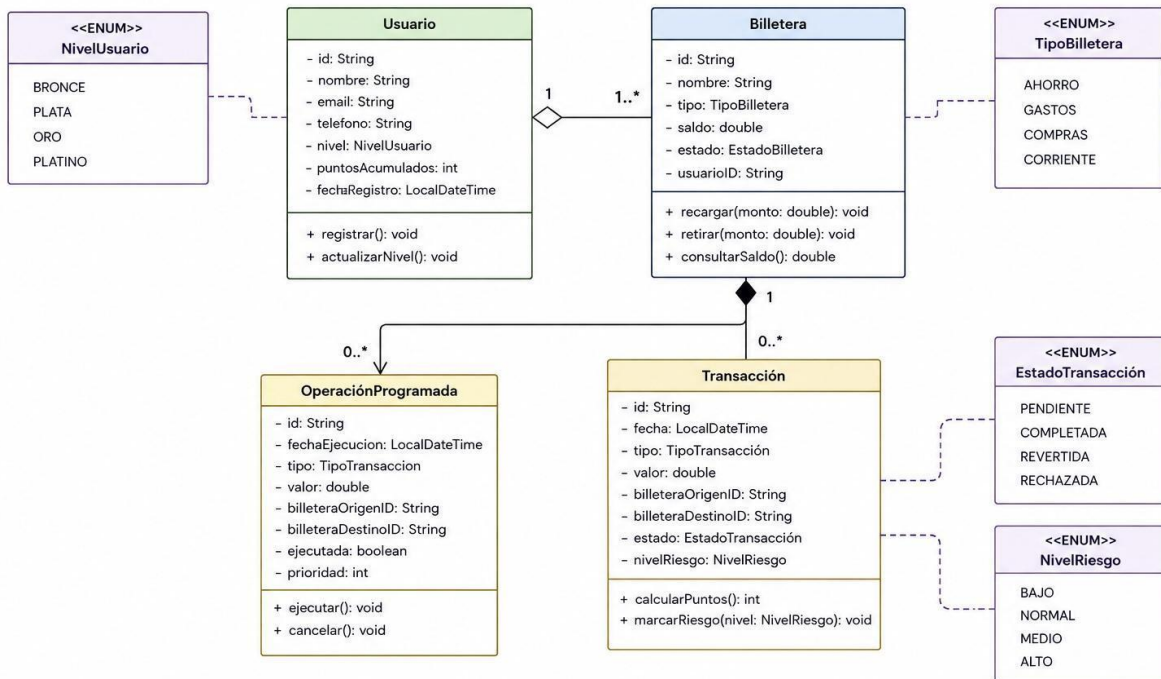
La enumeración NivelUsuario clasifica a los clientes según su actividad financiera y puntos acumulados, utilizando categorías como bronce, plata, oro y platino. Esta clasificación permite establecer beneficios, límites y recompensas personalizadas para cada tipo de usuario.

Por otro lado, TipoBilletera permite diferenciar las distintas finalidades financieras de las cuentas registradas dentro del sistema, mientras que EstadoTransaccion controla el flujo operativo de las transacciones. Finalmente, NivelRiesgo categoriza el comportamiento financiero asociado a cada operación, permitiendo fortalecer los mecanismos de seguridad y monitoreo implementados por la plataforma.

En conjunto, el dominio CORE proporciona una representación estructurada y organizada del funcionamiento interno del sistema financiero. La correcta definición de entidades, relaciones y reglas de negocio permite garantizar coherencia operativa, facilidad de mantenimiento y capacidad de escalabilidad para futuras ampliaciones del proyecto.

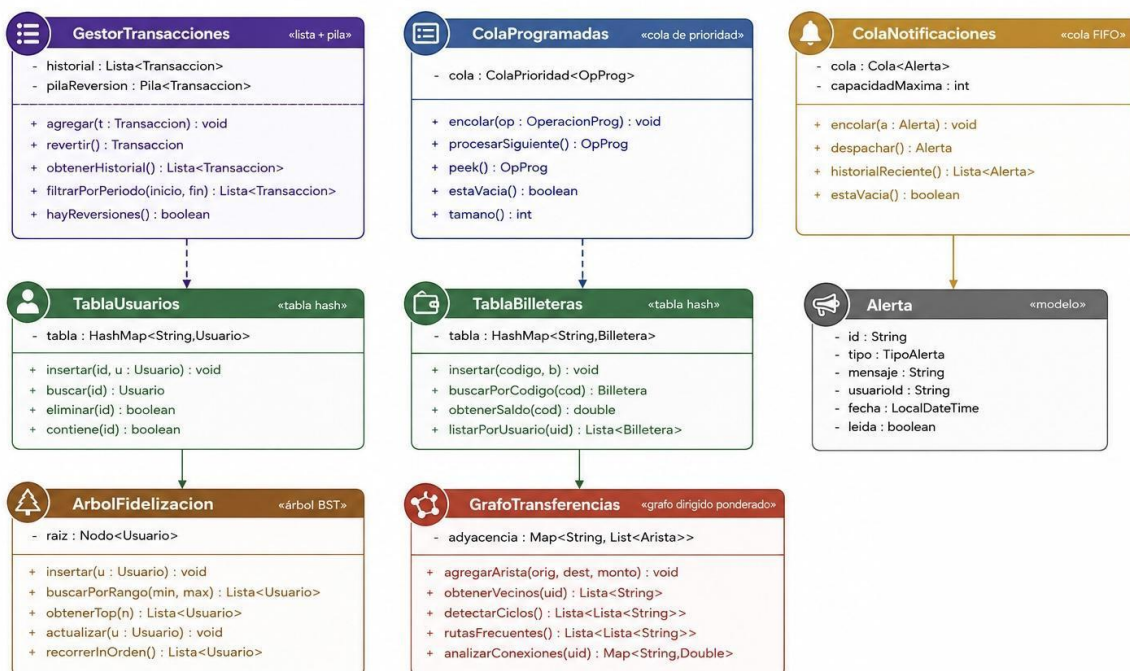
Diagrama Dominio

DOMINIO CORE



06. Estructuras de Datos Aplicadas

El proyecto utiliza diferentes estructuras de datos, cada una seleccionada de acuerdo con las necesidades funcionales del sistema.



Las listas son utilizadas para almacenar historiales de transacciones y reportes financieros, permitiendo mantener el orden cronológico de la información y facilitar consultas posteriores. -> Justificación: permiten conservar orden cronológico y recorrer el historial para reportes y filtros.

Las pilas se implementan para la reversión de operaciones financieras. Gracias a su funcionamiento LIFO, es posible deshacer transacciones recientes de manera eficiente con complejidad $O(1)$. -> Justificación: la operación más reciente puede revertirse en $O(1)$, imitando un historial de deshacer.

Las colas circulares permiten administrar notificaciones dentro del sistema, controlando el uso de memoria y eliminando automáticamente los registros más antiguos cuando se alcanza el límite establecido. -> Justificación: limita la memoria y descarta automáticamente la alerta más antigua cuando se llena.

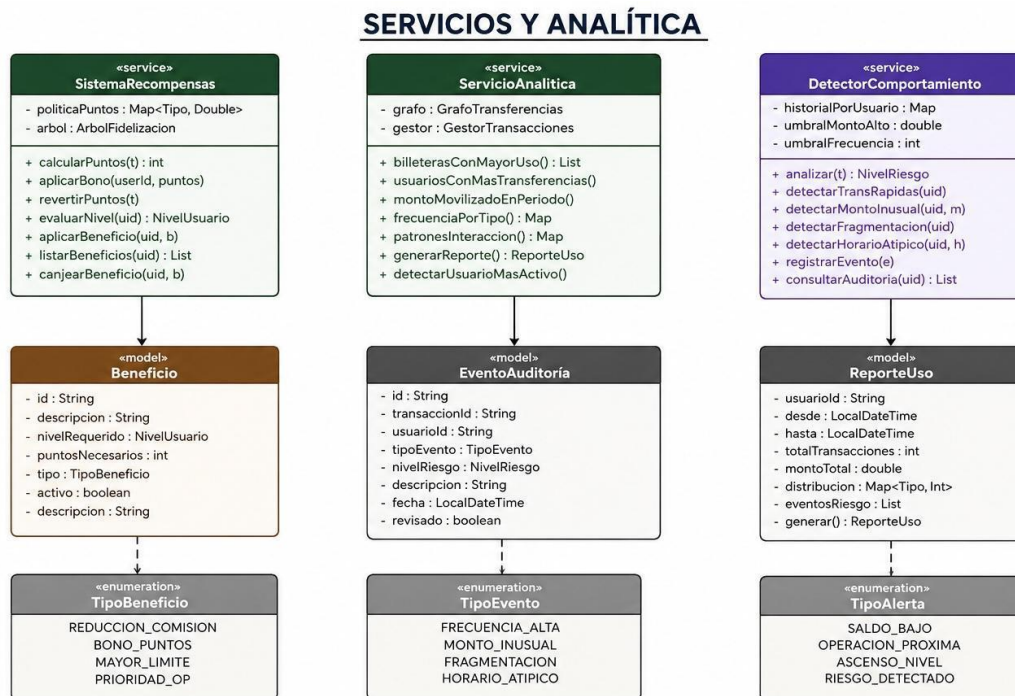
Las colas de prioridad gestionan las operaciones programadas, asegurando que las transacciones más urgentes sean procesadas primero según criterios temporales o de prioridad. -> Justificación: siempre procesa primero la operación programada más urgente.

Las tablas hash son empleadas para almacenar usuarios y billeteras debido a su capacidad de ofrecer búsquedas rápidas con complejidad aproximada $O(1)$, optimizando el acceso a la información. -> Justificación: aceleran búsquedas por identificador, consulta de saldo y actualizaciones frecuentes.

El árbol de fidelización, basado en BST, organiza usuarios según puntos acumulados y niveles de clasificación, permitiendo búsquedas eficientes por rangos. -> Justificación: facilita reportes ordenados, top de puntos y búsquedas por rango.

Finalmente, el grafo de transferencias modela las relaciones financieras entre usuarios y billeteras, permitiendo detectar patrones frecuentes, conexiones sospechosas y ciclos de transferencias. -> Justificación: permite analizar vecinos, montos transferidos, conexiones, ciclos y patrones de relación financiera.

07. Servicios y Analítica



La capa de Servicios y Analítica representa uno de los componentes más importantes dentro de la arquitectura del sistema ZIPLOC SAS, ya que concentra la lógica encargada de procesar información financiera, generar análisis estadísticos, administrar beneficios y supervisar comportamientos asociados al uso de la plataforma. Esta capa integra diferentes servicios especializados que trabajan de manera conjunta para optimizar la experiencia del usuario y fortalecer los mecanismos de control y seguridad.

El componente SistemaRecompensas administra el programa de fidelización implementado dentro de la plataforma. Su objetivo principal consiste en incentivar el uso frecuente del sistema mediante la asignación de puntos y beneficios asociados a la actividad financiera de los usuarios. Para ello, el servicio evalúa transacciones realizadas, frecuencia de uso, montos movilizados y niveles de clasificación del usuario dentro de la plataforma.

Este sistema utiliza estructuras como mapas y árboles de fidelización para organizar usuarios según sus puntos acumulados y nivel de actividad. Gracias a ello, es posible generar rankings, clasificaciones y consultas eficientes relacionadas con los usuarios más activos o con mayores

beneficios acumulados. Además, el sistema permite aplicar incentivos específicos como reducción de comisiones, bonos de puntos, ampliación de límites de transferencia y acceso prioritario a determinadas operaciones financieras.

Por otra parte, el componente ServicioAnalitica se encarga del procesamiento estadístico y análisis del comportamiento financiero dentro del sistema. Este servicio recopila información proveniente de transacciones, billeteras y relaciones entre usuarios para generar reportes detallados y métricas de uso.

Mediante la utilización de estructuras como grafos de transferencias y gestores de historial transaccional, el sistema puede identificar usuarios con mayor actividad financiera, detectar conexiones frecuentes entre cuentas, calcular frecuencias de operación y analizar patrones de comportamiento dentro de determinados periodos de tiempo. Estas capacidades permiten construir reportes de uso que facilitan la toma de decisiones y el monitoreo operativo del sistema.

Adicionalmente, la plataforma incorpora el componente DetectorComportamiento, encargado de implementar mecanismos orientados a la prevención de fraude y monitoreo de riesgos financieros. Este servicio analiza continuamente las operaciones realizadas por los usuarios utilizando umbrales de validación, historiales de actividad y patrones de comportamiento previamente definidos.

Entre las principales funcionalidades implementadas se encuentran la detección de transferencias rápidas entre múltiples cuentas, análisis de montos inusuales, identificación de operaciones fuera de horarios normales y detección de posibles intentos de fragmentación de transacciones. Estas validaciones permiten clasificar el nivel de riesgo asociado a cada operación y generar alertas preventivas dentro del sistema.

Los eventos sospechosos detectados son almacenados mediante registros de auditoría, garantizando trazabilidad completa sobre incidentes financieros relevantes. Cada evento conserva información relacionada con el usuario involucrado, tipo de comportamiento detectado, nivel de riesgo, fecha de ocurrencia y descripción detallada de la anomalía identificada. Esto facilita procesos posteriores de revisión, análisis de seguridad y auditorías internas.

Finalmente, esta capa también incorpora enumeraciones especializadas como TipoBeneficio, TipoEvento y TipoAlerta, las cuales permiten categorizar de manera organizada los distintos beneficios, eventos de auditoría y alertas generadas por el sistema. Esta estandarización mejora la claridad del modelo de dominio y facilita la administración de reglas de negocio relacionadas con procesos analíticos y de monitoreo financiero.

08. Endpoints y Funcionalidades

La plataforma dispone de diversos endpoints REST orientados a la administración integral del sistema financiero:

- /api/usuarios
- /api/billeteras
- /api/transacciones
- /api/programadas
- /api/recompensas
- /api/notificaciones
- /api/analitica

A través de estos servicios se implementan funcionalidades como gestión de usuarios, administración de billeteras, operaciones financieras, reversión de transacciones, programación automática de pagos, notificaciones, recompensas y procesos de analítica financiera.

09. Requisitos No Funcionales

El sistema fue diseñado considerando diferentes requisitos no funcionales orientados a garantizar calidad y rendimiento.

La modularidad permite mantener una estructura organizada y escalable. La eficiencia se logra mediante el uso adecuado de estructuras de datos optimizadas para cada operación. La validación de datos garantiza consistencia en las transacciones y evita errores operativos.

Asimismo, el manejo de excepciones y errores fortalece la estabilidad del sistema, mientras que la consistencia de la información asegura confiabilidad en las operaciones financieras.

10. Conclusiones

El desarrollo de ZIPLOC SAS demuestra la importancia de las estructuras de datos dentro de sistemas financieros modernos. La correcta integración de listas, pilas, colas, árboles, tablas hash y grafos permite construir soluciones eficientes, organizadas y escalables.

Además, la implementación de arquitectura por capas y servicios de analítica fortalece la capacidad del sistema para procesar información financiera y detectar comportamientos sospechosos en tiempo real.

Desde una perspectiva académica, el proyecto evidencia cómo los conceptos teóricos de estructuras de datos pueden aplicarse en problemas reales relacionados con plataformas fintech, aportando eficiencia operativa, seguridad y capacidad analítica.

Referencias

Spring Boot. (2026). Documentación oficial de Spring Boot. <https://spring.io/projects/spring-boot>

Goodrich, M., Tamassia, R., & Goldwasser, M. (2014). Data Structures and Algorithms in Java. Wiley.

Cormen, T., Leiserson, C., Rivest, R., & Stein, C. (2009). Introduction to Algorithms. MIT Press.